

Seasonal Flu Vaccine Prediction

Project: Phase 3

Author: [Sharon Kipruto]

Business Understanding

Seasonal flu vaccination uptake is influenced by a combination of demographic factors, health status, prior medical behavior and personal beliefs about vaccines. Public health organizations need data-driven ways to identify individuals who are less likely to get vaccinated so that outreach and education efforts can be more effectively targeted.

Understanding vaccination behavior helps public health organizations design targeted interventions and improve vaccine adoption during pandemics.

Problem Statement and Stakeholder

Stakeholder: Public Health Organizations

Public health organizations often feel limited in resources when promoting vaccines. Being able to predict whether individuals receive the vaccines allows officials to target education and outreach.

objective

This is a binary classification problem where the target variable is whether a respondent received the seasonal flu vaccine based on their demographics and how to support public health interventions.

TARGET VARIABLE: Seasonal Flu Vaccine (0-NOT vaccinated, 1-vaccinated)

Metric Goal:

Our primary evaluation metric will be Recall to minimize false negatives (i.e., individuals who did not receive the vaccine but should have).

A false negative represents a person the model predicts will get vaccinated but doesn't; in public health, missing these individuals means failing to prevent a potential infection. We would like to identify as many vaccinated individuals as possible to improve public health outcomes.

Our secondary metric will be F1-score to balance precision and recall. This ensures model is both accurate and sensitive.

Other metrics like accuracy and precision will be reported for completeness but will not be the primary decision criterion.

Dataset Overview

The dataset used in this project is the National 2009 H1N1 Flu Survey [NHFS](#) and includes demographic information, health behaviors, opinions about vaccines, and risk factors.

- Rows represent individual survey respondents
- Features include demographics, health status, beliefs, and behaviors
- Targets indicate whether the respondent received a flu vaccine

Feature Categories

The dataset contains a wide range of features describing each respondent.
For clarity and better interpretation, the features can be grouped into the following categories:

1. Demographic Features

These describe basic personal characteristics of respondents:

- Age group
- Sex
- Race
- Education level
- Marital status
- Employment status
- Household income
- Number of household members

2. Health Status

These features describe the general health condition of respondents:

- Self-reported health status
- Presence of chronic conditions
- Health insurance coverage

3. Medical History

These variables capture past medical and healthcare-related behavior:

- History of receiving flu vaccines
- Doctor visits
- Use of healthcare services

4. Opinions and Beliefs About Vaccines

These features reflect personal perceptions and attitudes:

- Perceived effectiveness of vaccines
- Concerns about vaccine safety
- Level of trust in medical professionals
- Beliefs about vaccine risks

5. Behavioral and Risk Factors

These variables describe lifestyle and exposure-related behavior:

- Frequency of social contact
- Exposure risk due to work or environment
- Preventive health behaviors

In [300]:

```
#IMPORT LIBRARIES

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# For modelling
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, recall_score, f1_score
, ConfusionMatrixDisplay
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, accuracy_score, recall_score, f1_score
from sklearn.multioutput import MultiOutputClassifier
from sklearn.tree import DecisionTreeClassifier

# To ignore warnings
import warnings
warnings.filterwarnings('ignore')

```

DATA PREPARATION AND PREPROCESSING

The steps taken here are:

1. merging datasets(feature and target datasets) and handling missing values
2. encoding categorical data for easy machine learning analysis and ensure they are all numerical
3. selecting target and features
4. Train test split

In [301]:

```

features_df = pd.read_csv('DATA/training_set_features.csv')
target_df = pd.read_csv('DATA/training_set_labels.csv')

```

In [302]:

```
features_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26707 entries, 0 to 26706
Data columns (total 36 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   respondent_id                        26707 non-null  int64
 1   h1n1_concern                        26615 non-null  float64
 2   h1n1_knowledge                      26591 non-null  float64
 3   behavioral_antiviral_meds           26636 non-null  float64
 4   behavioral_avoidance                26499 non-null  float64
 5   behavioral_face_mask                26688 non-null  float64
 6   behavioral_wash_hands               26665 non-null  float64
 7   behavioral_large_gatherings         26620 non-null  float64
 8   behavioral_outside_home             26625 non-null  float64
 9   behavioral_touch_face               26579 non-null  float64
10   doctor_recc_h1n1                   24547 non-null  float64
11   doctor_recc_seasonal                24547 non-null  float64
12   chronic_med_condition               25736 non-null  float64
13   child_under_6_months               25887 non-null  float64
14   health_worker                      25903 non-null  float64
15   health_insurance                   14433 non-null  float64
16   opinion_h1n1_vacc_effective          26316 non-null  float64
17   opinion_h1n1_risk                   26319 non-null  float64
18   opinion_h1n1_sick_from_vacc         26312 non-null  float64
19   opinion_seas_vacc_effective          26245 non-null  float64
20   opinion_seas_risk                   26193 non-null  float64
21   opinion_seas_sick_from_vacc         26170 non-null  float64
22   age_group                          26707 non-null  object
23   education                          25300 non-null  object
24   race                              26707 non-null  object
25   sex                              26707 non-null  object
26   income_poverty                     22284 non-null  object
27   marital_status                     25299 non-null  object
28   rent_or_own                       24665 non-null  object
29   employment_status                 25244 non-null  object
30   hhs_geo_region                     26707 non-null  object
31   census_msa                        26707 non-null  object
32   household_adults                   26458 non-null  float64
33   household_children                 26458 non-null  float64

```

```
34 employment_industry      13377 non-null object
35 employment_occupation     13237 non-null object
dtypes: float64(23), int64(1), object(12)
memory usage: 7.3+ MB
```

In [303]:

```
target_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26707 entries, 0 to 26706
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   respondent_id    26707 non-null  int64
1   h1n1_vaccine     26707 non-null  int64
2   seasonal_vaccine 26707 non-null  int64
dtypes: int64(3)
memory usage: 626.1 KB
```

The two datasets have equal number of rows and share a common column 'respondent_id' which can be used to merge them. The target variables are 'h1n1_vaccine' and 'seasonal_vaccine'. We will choose one target variable for prediction.

In [304]:

```
print(features_df.shape)
print(target_df.shape)
```

```
(26707, 36)
(26707, 3)
```

In [305]:

```
#merging dataframes on respondent_id
df = features_df.merge(target_df, on='respondent_id')

#check new shape
print(f"Combined Shape: {df.shape}")

#check first 5 rows
df.head()
```

```
Combined Shape: (26707, 38)
```

Out[305]:

	respondent_id	h1n1_concern	h1n1_knowledge	behavioral_antiviral_meds	behavioral_avoidance	behavioral_face_mask	beh
0	0	1.0	0.0	0.0	0.0	0.0	
1	1	3.0	2.0	0.0	1.0	0.0	
2	2	1.0	1.0	0.0	1.0	0.0	
3	3	1.0	1.0	0.0	1.0	0.0	
4	4	2.0	1.0	0.0	1.0	0.0	

5 rows x 38 columns

In [306]:

```
#checking info of combined dataframe
```

```
#checking info of combined dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 26707 entries, 0 to 26706
```

```
Data columns (total 38 columns):
```

#	Column	Non-Null Count	Dtype
0	respondent_id	26707 non-null	int64
1	h1n1_concern	26615 non-null	float64
2	h1n1_knowledge	26591 non-null	float64
3	behavioral_antiviral_meds	26636 non-null	float64
4	behavioral_avoidance	26499 non-null	float64
5	behavioral_face_mask	26688 non-null	float64
6	behavioral_wash_hands	26665 non-null	float64
7	behavioral_large_gatherings	26620 non-null	float64
8	behavioral_outside_home	26625 non-null	float64
9	behavioral_touch_face	26579 non-null	float64
10	doctor_recc_h1n1	24547 non-null	float64
11	doctor_recc_seasonal	24547 non-null	float64
12	chronic_med_condition	25736 non-null	float64
13	child_under_6_months	25887 non-null	float64
14	health_worker	25903 non-null	float64
15	health_insurance	14433 non-null	float64
16	opinion_h1n1_vacc_effective	26316 non-null	float64
17	opinion_h1n1_risk	26319 non-null	float64
18	opinion_h1n1_sick_from_vacc	26312 non-null	float64
19	opinion_seas_vacc_effective	26245 non-null	float64
20	opinion_seas_risk	26193 non-null	float64
21	opinion_seas_sick_from_vacc	26170 non-null	float64
22	age_group	26707 non-null	object
23	education	25300 non-null	object
24	race	26707 non-null	object
25	sex	26707 non-null	object
26	income_poverty	22284 non-null	object
27	marital_status	25299 non-null	object
28	rent_or_own	24665 non-null	object
29	employment_status	25244 non-null	object
30	hhs_geo_region	26707 non-null	object
31	census_msa	26707 non-null	object
32	household_adults	26458 non-null	float64
33	household_children	26458 non-null	float64
34	employment_industry	13377 non-null	object
35	employment_occupation	13237 non-null	object
36	h1n1_vaccine	26707 non-null	int64
37	seasonal_vaccine	26707 non-null	int64

```
dtypes: float64(23), int64(3), object(12)
```

```
memory usage: 7.7+ MB
```

```
In [307]:
```

```
df.describe()
```

```
Out[307]:
```

	respondent_id	h1n1_concern	h1n1_knowledge	behavioral_antiviral_meds	behavioral_avoidance	behavioral_face_mask
count	26707.000000	26615.000000	26591.000000	26636.000000	26499.000000	26688.000000
mean	13353.000000	1.618486	1.262532	0.048844	0.725612	0.068982
std	7709.791156	0.910311	0.618149	0.215545	0.446214	0.253429
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	6676.500000	1.000000	1.000000	0.000000	0.000000	0.000000
50%	13353.000000	2.000000	1.000000	0.000000	1.000000	0.000000
75%	20029.500000	2.000000	2.000000	0.000000	1.000000	0.000000
max	26706.000000	3.000000	2.000000	1.000000	1.000000	1.000000

```
8 rows x 6 columns
```

In [308]:

```
#check for missing values and sort in descending order  
df.isna().sum().to_frame().sort_values(0,ascending = False).assign(Dtype=df.dtypes)
```

Out[308]:

	0	Dtype
employment_occupation	13470	object
employment_industry	13330	object
health_insurance	12274	float64
income_poverty	4423	object
doctor_recc_h1n1	2160	float64
doctor_recc_seasonal	2160	float64
rent_or_own	2042	object
employment_status	1463	object
marital_status	1408	object
education	1407	object
chronic_med_condition	971	float64
child_under_6_months	820	float64
health_worker	804	float64
opinion_seas_sick_from_vacc	537	float64
opinion_seas_risk	514	float64
opinion_seas_vacc_effective	462	float64
opinion_h1n1_sick_from_vacc	395	float64
opinion_h1n1_vacc_effective	391	float64
opinion_h1n1_risk	388	float64
household_adults	249	float64
household_children	249	float64
behavioral_avoidance	208	float64
behavioral_touch_face	128	float64
h1n1_knowledge	116	float64
h1n1_concern	92	float64
behavioral_large_gatherings	87	float64
behavioral_outside_home	82	float64
behavioral_antiviral_meds	71	float64
behavioral_wash_hands	42	float64
behavioral_face_mask	19	float64
h1n1_vaccine	0	int64
respondent_id	0	int64
census_msa	0	object
hhs_geo_region	0	object
sex	0	object
race	0	object
age_group	0	object
seasonal_vaccine	0	int64

In [309]:

```
#sort percentage of missing values and display data types
(df.isna().mean() * 100).sort_values(ascending=False).to_frame(name='Percentage_missing')
).assign(Dtype=df.dtypes).style.bar(subset=['Percentage_missing'], color="#67eef0")
```

Out[309]:

	Percentage_missing	Dtype
employment_occupation	50.436215	object
employment_industry	49.912008	object
health_insurance	45.957989	float64
income_poverty	16.561201	object
doctor_recc_h1n1	8.087767	float64
doctor_recc_seasonal	8.087767	float64
rent_or_own	7.645936	object
employment_status	5.477965	object
marital_status	5.272026	object
education	5.268282	object
chronic_med_condition	3.635751	float64
child_under_6_months	3.070356	float64
health_worker	3.010447	float64
opinion_seas_sick_from_vacc	2.010709	float64
opinion_seas_risk	1.924589	float64
opinion_seas_vacc_effective	1.729884	float64
opinion_h1n1_sick_from_vacc	1.479013	float64
opinion_h1n1_vacc_effective	1.464036	float64
opinion_h1n1_risk	1.452803	float64
household_adults	0.932340	float64
household_children	0.932340	float64
behavioral_avoidance	0.778822	float64
behavioral_touch_face	0.479275	float64
h1n1_knowledge	0.434343	float64
h1n1_concern	0.344479	float64
behavioral_large_gatherings	0.325757	float64
behavioral_outside_home	0.307036	float64
behavioral_antiviral_meds	0.265848	float64
behavioral_wash_hands	0.157262	float64
behavioral_face_mask	0.071142	float64
h1n1_vaccine	0.000000	int64
respondent_id	0.000000	int64
census_msa	0.000000	object
hhs_geo_region	0.000000	object
sex	0.000000	object
race	0.000000	object
age_group	0.000000	object
seasonal_vaccine	0.000000	int64

Categorical columns with missing values

from the recent missing values analysis, we can see that `employment_industry`, `employment_occupation` and `health_insurance` have significant missing values. We will drop these columns for simplicity and since they are not important for our analysis.

In [310]:

```
#drop columns with significant missing values
df=df.drop(columns=['employment_industry', 'employment_occupation', 'health_insurance'])

print("Dropped columns: employment_industry, employment_occupation, health_insurance")
print(f"New shape after dropping columns: {df.shape}")
```

```
Dropped columns: employment_industry, employment_occupation, health_insurance
New shape after dropping columns: (26707, 35)
```

Columns like `income_poverty`, `rent_or_own`, `employment_occupation`, `employment_status`, `education` and `marital_status` have a moderate amount of missing data. Since these are "Object" (text) types, you can't use a mean or median. For these we will fill in the missing values with 'Unknown' so as we are not making any assumptions about the person's life.

In [311]:

```
#fill in missing values with 'Unknown' for categorical columns
cat_cols=['income_poverty', 'rent_or_own', 'marital_status', 'employment_status', 'education']
for col in cat_cols:
    df[col] = df[col].fillna('Unknown')

print("Filled missing values for categorical columns with 'Unknown'")
```

```
Filled missing values for categorical columns with 'Unknown'
```

For columns like `h1n1_knowledge`, `option_h1n1_risk` or `opinion_seas_risk` the missingness is very low. we will impute with the mode of the respective columns.

In [312]:

```
print(df['h1n1_knowledge'].value_counts())
print()
print(df['opinion_seas_risk'].value_counts())
print()
print(df['opinion_h1n1_risk'].value_counts())
```

```
h1n1_knowledge
1.0    14598
2.0     9487
0.0     2506
Name: count, dtype: int64
```

```
opinion_seas_risk
2.0     8954
4.0     7630
1.0     5974
5.0     2958
3.0       677
Name: count, dtype: int64
```

```
opinion_h1n1_risk
2.0     9919
1.0     8139
4.0     5394
5.0     1750
3.0     1117
Name: count, dtype: int64
```


In [313]:

```
# #impute with the mode for columns with low missingness
low_missing_cols = [
    'h1n1_knowledge', 'opinion_seas_risk', 'opinion_h1n1_risk',
    'doctor_recc_h1n1', 'doctor_recc_seasonal']

#mode imputation
imputer=SimpleImputer(strategy='most_frequent')

#fit and transform the low missing columns
df[low_missing_cols]=imputer.fit_transform(df[low_missing_cols])

# #create a loop to fill missing values with mode
# for cols in low_missing_cols:
#     df[cols]=df[cols].fillna(df[cols].mode()[0])
```

now we have handled missing values in categorical columns. Next, we will handle missing values in numerical columns.

Numerical columns with missing values

For numerical columns with missing values, we can impute the missing values with the mean or median of the respective columns. We will use the mean for this analysis.

In [314]:

```
num_cols = df.select_dtypes(include=['number']).columns[
    df.select_dtypes(include=['number']).isna().any()]

#create a loop to fill missing values with mode
for col in num_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print(f'filled missing values for: {list(num_cols)}')

# num_imputer = SimpleImputer(strategy='median')
# df[num_cols] = num_imputer.fit_transform(df[num_cols])
```

filled missing values for: ['h1n1_concern', 'behavioral_antiviral_meds', 'behavioral_avoidance', 'behavioral_face_mask', 'behavioral_wash_hands', 'behavioral_large_gatherings', 'behavioral_outside_home', 'behavioral_touch_face', 'chronic_med_condition', 'child_under_6_months', 'health_worker', 'opinion_h1n1_vacc_effective', 'opinion_h1n1_sick_from_vacc', 'opinion_seas_vacc_effective', 'opinion_seas_sick_from_vacc', 'household_adults', 'household_children']

In [315]:

```
#recheck missing values
(df.isna().mean() * 100).sort_values(ascending=False).to_frame(name='Percentage_missing')
.assign(Dtype=df.dtypes).style.bar(subset=['Percentage_missing'], color="#67eef0")
```

Out[315]:

	Percentage_missing	Dtype
respondent_id	0.000000	int64
marital_status	0.000000	object
opinion_seas_sick_from_vacc	0.000000	float64
age_group	0.000000	object
education	0.000000	object
race	0.000000	object
sex	0.000000	object
income_poverty	0.000000	object

rent_or_own	Percentage_missing	object Dtype
opinion_seas_vacc_effective	0.000000	float64
employment_status	0.000000	object
hhs_geo_region	0.000000	object
census_msa	0.000000	object
household_adults	0.000000	float64
household_children	0.000000	float64
h1n1_vaccine	0.000000	int64
opinion_seas_risk	0.000000	float64
opinion_h1n1_sick_from_vacc	0.000000	float64
h1n1_concern	0.000000	float64
behavioral_outside_home	0.000000	float64
h1n1_knowledge	0.000000	float64
behavioral_antiviral_meds	0.000000	float64
behavioral_avoidance	0.000000	float64
behavioral_face_mask	0.000000	float64
behavioral_wash_hands	0.000000	float64
behavioral_large_gatherings	0.000000	float64
behavioral_touch_face	0.000000	float64
opinion_h1n1_risk	0.000000	float64
doctor_recc_h1n1	0.000000	float64
doctor_recc_seasonal	0.000000	float64
chronic_med_condition	0.000000	float64
child_under_6_months	0.000000	float64
health_worker	0.000000	float64
opinion_h1n1_vacc_effective	0.000000	float64
seasonal_vaccine	0.000000	int64

we have handled missing values in both categorical and numerical columns.

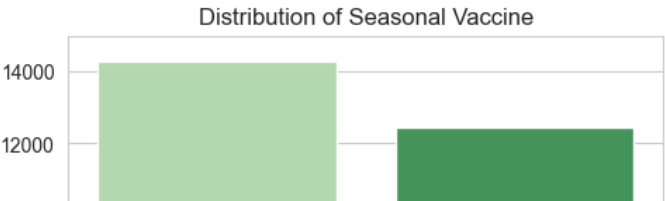
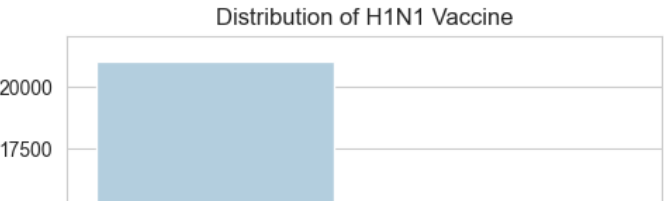
EXPLORATORY DATA ANALYSIS (EDA)

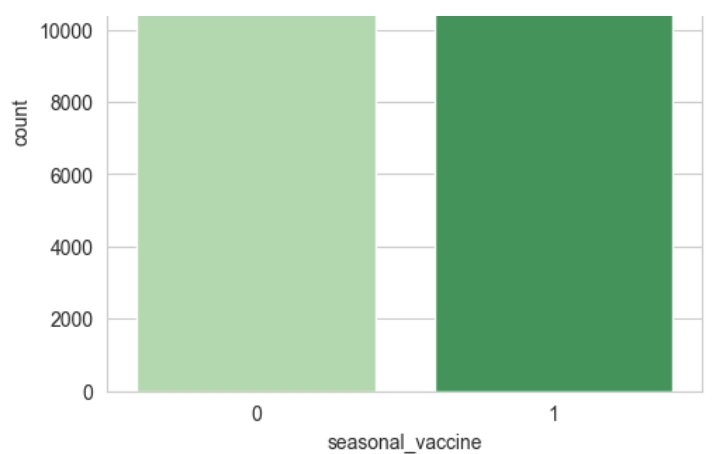
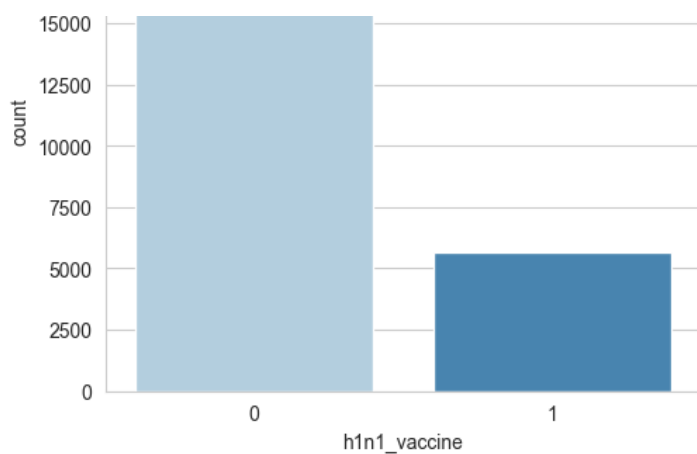
We will explore the distribution of the target variables and identify correlations between features and vaccination status.

In [316]:

```
# Visualize target variable distribution
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
sns.countplot(x='h1n1_vaccine', data=df, ax=ax[0], palette='Blues')
ax[0].set_title('Distribution of H1N1 Vaccine')

sns.countplot(x='seasonal_vaccine', data=df, ax=ax[1], palette='Greens')
ax[1].set_title('Distribution of Seasonal Vaccine')
plt.show()
plt.savefig('./Visualizations/vaccine_distribution.png')
```





<Figure size 640x480 with 0 Axes>

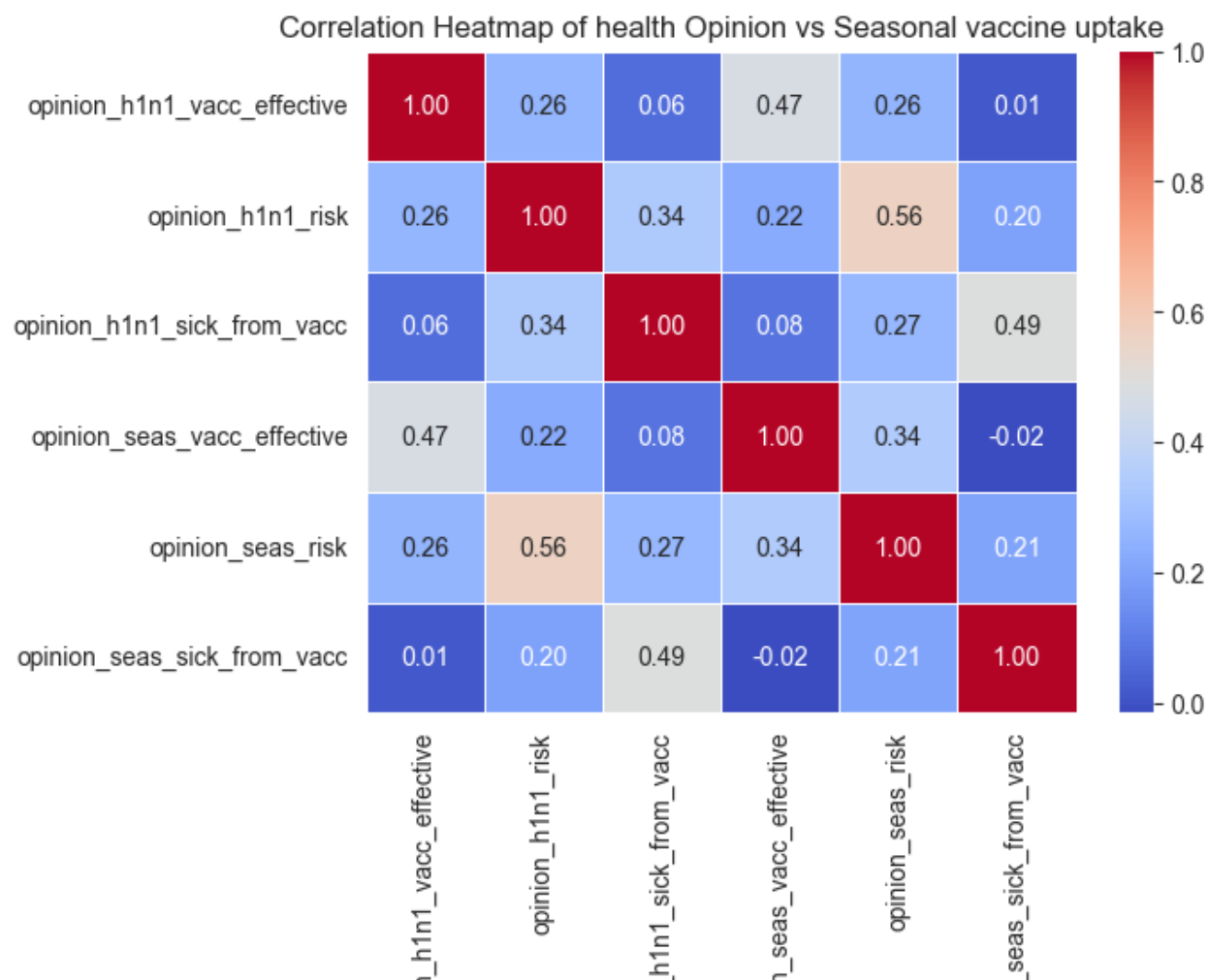
correlation of Health opinions.

In this section we analyze how opinions about the flu and vaccines correlate with vaccination uptake. Understanding these correlations can help public health officials design better communication strategies to improve vaccination rates.

In [317]:

```
opiniopn_cols=[
    'opinion_h1n1_vacc_effective', 'opinion_h1n1_risk',
    'opinion_h1n1_sick_from_vacc', 'opinion_seas_vacc_effective',
    'opinion_seas_risk', 'opinion_seas_sick_from_vacc'
]

corr_matrix = df[opiniopn_cols].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidth=0.5)
plt.title('Correlation Heatmap of health Opinion vs Seasonal vaccine uptake')
plt.show()
plt.savefig('./Visualizations/opinion_correlation_heatmap.png')
```



<Figure size 640x480 with 0 Axes>

from the above correlation we see that `opinion_seas_risk` and `opinion_seas_vacc_effective` have the highest positive correlation with `seasonal_vaccine` uptake.

This suggests that individuals who perceive a higher risk from the seasonal flu and believe in the effectiveness of the seasonal vaccine are more likely to get vaccinated. Public health campaigns could focus on educating the public about the risks of the flu and the benefits of vaccination to improve uptake rates.

Selecting target and feature variables

This project will focus on `seasonal_flu_vaccine` ,because: it was a novel pandemic like COVID-19 it is binary hence perfect for a classification task it is good for understanding vaccine uptake/hesitancy

In [318]:

```
# Features (drop the targets and the ID)
X = df.drop(columns=['respondent_id', 'h1n1_vaccine', 'seasonal_vaccine'])

# Targets (we have two!)
y = df[['seasonal_vaccine']]
```

In [319]:

```
y.value_counts(normalize=True)
```

Out[319]:

```
seasonal_vaccine
0                0.534392
1                0.465608
Name: proportion, dtype: float64
```

Encoding Categorical Variables

In [320]:

```
# Identify categorical and numerical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
numeric_cols = X.select_dtypes(exclude=['object']).columns.tolist()

print(f"Categorical columns: {len(categorical_cols)}")
print(f"Numerical columns: {len(numeric_cols)}")
```

```
Categorical columns: 10
Numerical columns: 22
```

In [321]:

```
#encode the categorical variables using one-hot encoding
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
X_encoded = pd.DataFrame(encoder.fit_transform(X[categorical_cols]))
X_encoded.index = X.index # Align indices after encoding
```

In [322]:

```
#merge numerical and encoded categorical features
X = pd.concat([X[numeric_cols], X_encoded], axis=1)

print(f"Total features after encoding: {X.shape[1]}")

#ensure all column names are strings
```

```
X.columns = X.columns.astype(str)

#check first 5 rows
X.head()
```

Total features after encoding: 65

Out[322]:

	h1n1_concern	h1n1_knowledge	behavioral_antiviral_meds	behavioral_avoidance	behavioral_face_mask	behavioral_wash_ha
0	1.0	0.0	0.0	0.0	0.0	
1	3.0	2.0	0.0	1.0	0.0	
2	1.0	1.0	0.0	1.0	0.0	
3	1.0	1.0	0.0	1.0	0.0	
4	2.0	1.0	0.0	1.0	0.0	

5 rows x 65 columns



Train-Test Split

The daata was split into training and testing sets to evaluate how well the model generalizes to unseen data and reduce risk of overfitting.

In [323]:

```
#splitting the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,stratify=y)

#stratify=y preserves class balance
#random_state=42 ensures reproducibility
```

In [324]:

```
print("Dataset split summary:")
print(f"Training set:      {X_train.shape[0]:6,} samples ({X_train.shape[0]/len(X)*100:.1f}%
)")
print(f"Validation set: {X_test.shape[0]:6,} samples ({X_test.shape[0]/len(X)*100:.1f}%)"
)
print(f"Features:          {X_train.shape[1]:6} features")
```

Dataset split summary:
Training set: 21,365 samples (80.0%)
Validation set: 5,342 samples (20.0%)
Features: 65 features

Classification Models

Multiple classification models were trained and evaluated, including:

- Logistic Regression
- Decision Tree
- Random Forest

These models were chosen because they are interpretable and appropriate for public health applications. For each model we will evaluate metrics like **Accuracy, Recall,precision,F1-score** and display their **confssion-matrices** `Recall` is more important as it is good for minimizing false negatives.

MODEL 1: baseline Model

LOGISTIC REGRESSION MODEL

LOGISTIC REGRESSION MODEL

fitting the baseline logistic regression model to the training data and evaluating its performance on the test data.
why logistic regression?

- Simple and interpretable
- Fast to train
- Works well when relationships are approximately linear
- Good baseline for comparison
- Calculates probability of each class based on weighted feature combinations.

The baseline is selected to establish minimum performance benchmark.

In [325]:

```
#scaling features
scalar = StandardScaler()
X_scaled = scalar.fit_transform(X_train)
X_test_scaled = scalar.transform(X_test)

#model on seasonal vaccine target
logReg_seasonal=LogisticRegression(max_iter=1000,class_weight='balanced',random_state=42)

#fit the model to training data
logReg_seasonal.fit(X_scaled, y_train['seasonal_vaccine'])

#predict on test set
train_pred_seasonal_lr = logReg_seasonal.predict(X_scaled)
test_pred_seasonal_lr = logReg_seasonal.predict(X_test_scaled)

#check accuracy seasonal
train_accuracy_seasonal_lr = accuracy_score(y_train['seasonal_vaccine'],train_pred_seasonal_lr)
test_accuracy_seasonal_lr = accuracy_score(y_test['seasonal_vaccine'],test_pred_seasonal_lr)

#check recall seasonal
train_recall_seasonal_lr = recall_score(y_train['seasonal_vaccine'],train_pred_seasonal_lr)
test_recall_seasonal_lr = recall_score(y_test['seasonal_vaccine'],test_pred_seasonal_lr)

#check f1 seasonal
train_f1_seasonal_lr = f1_score(y_train['seasonal_vaccine'],train_pred_seasonal_lr)
test_f1_seasonal_lr = f1_score(y_test['seasonal_vaccine'],test_pred_seasonal_lr)
```

In [326]:

```
metrics_lr=pd.DataFrame({
    "Train Accuracy": [train_accuracy_seasonal_lr],
    "Test Accuracy": [test_accuracy_seasonal_lr],
    "Gap": [train_accuracy_seasonal_lr - test_accuracy_seasonal_lr],

    "Train Recall": [train_recall_seasonal_lr],
    "Test Recall": [test_recall_seasonal_lr],

    "Train F1 Score": [train_f1_seasonal_lr],
    "Test F1 Score": [test_f1_seasonal_lr]
}).T

metrics_lr.columns = ['Logistic Regression - Seasonal']
metrics_lr
```

Out[326]:

Logistic Regression - Seasonal	
Train Accuracy	0.775099
Test Accuracy	0.770000

Test Accuracy	0.779858
Logistic Regression - Seasonal Vaccine	
Gap	0.000758
Train Recall	0.765883
Test Recall	0.760354
Train F1 Score	0.760265
Test F1 Score	0.762808

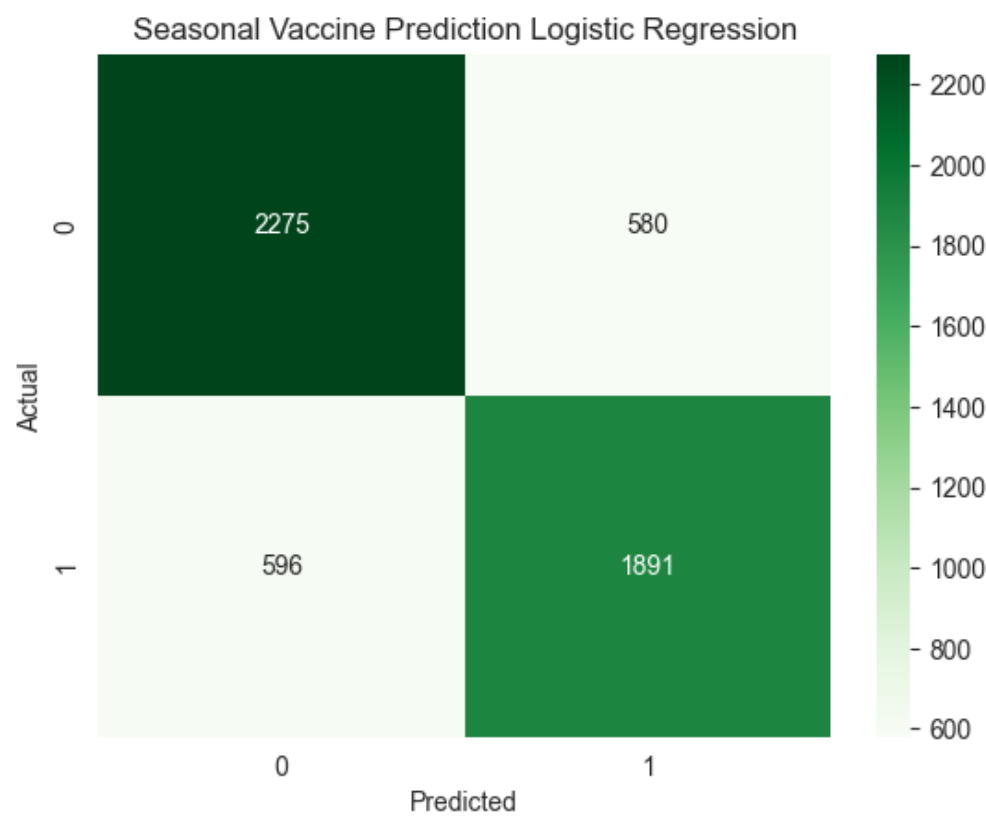
confusion matrix heatmaps

In [327]:

```
# Plot logReg confusion matrix for seasonal vaccine
sns.heatmap(confusion_matrix(y_test['seasonal_vaccine'], test_pred_seasonal_lr),
            annot=True, fmt='d', cmap='Greens')

# Set labels using plt instead of ax
plt.title('Seasonal Vaccine Prediction Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

plt.savefig('./Visualizations/LogReg_confusion_matrix.png')
```



<Figure size 640x480 with 0 Axes>

Logistic regression model summary and insights

The model correctly classified 2275 non-vaccinated and 1891 vaccinated patients.

Errors include 580 false positives and 596 false negatives, showing a slight tendency to under-predict vaccination.

Overall: While accuracy is high, confusion matrices reveal subtle misclassification patterns that are important for public health interpretation.

overfitting or underfitting analysis:

Logistic regression seasonal prediction accuracy:

Training Accuracy: 0.7751

Test Accuracy: 0.7799

The Gap: -0.0048

The gap for both is extremely small, indicating that the model generalizes well to unseen data without significant overfitting or underfitting.

#

MODEL 2

DECISION TREE CLASSIFIER MODEL

In [328]:

```
dt_seasonal= DecisionTreeClassifier(  
    class_weight='balanced', random_state=42,max_depth=5)  
  
dt_seasonal.fit(X_train, y_train['seasonal_vaccine'])  
  
#fit the model to training data for seasonal vaccine  
dt_seasonal.fit(X_train, y_train['seasonal_vaccine'])  
  
# predict on test set for seasonal vaccine  
train_preds_seasonal_dt = dt_seasonal.predict(X_train)  
test_pred_seasonal_dt=dt_seasonal.predict(X_test)  
  
seasonal_train_acc_dt = accuracy_score(y_train['seasonal_vaccine'], train_preds_seasonal_dt)  
seasonal_test_acc_dt = accuracy_score(y_test['seasonal_vaccine'], test_pred_seasonal_dt)  
  
#check recall seasonal  
seasonal_train_recall_dt = recall_score(y_train['seasonal_vaccine'],train_preds_seasonal_dt)  
seasonal_test_recall_dt = recall_score(y_test['seasonal_vaccine'],test_pred_seasonal_dt)  
  
#check f1 seasonal  
seasonal_train_f1_dt = f1_score(y_train['seasonal_vaccine'],train_preds_seasonal_dt)  
seasonal_test_f1_dt = f1_score(y_test['seasonal_vaccine'],test_pred_seasonal_dt)
```

In [329]:

```
metrics_dt=pd.DataFrame({  
    "Train Accuracy": [seasonal_train_acc_dt],  
    "Test Accuracy": [seasonal_test_acc_dt],  
    "Gap": [seasonal_train_acc_dt - seasonal_test_acc_dt],  
  
    "Train Recall": [seasonal_train_recall_dt],  
    "Test Recall": [seasonal_test_recall_dt],  
  
    "Train F1 Score": [seasonal_train_f1_dt],  
    "Test F1 Score": [seasonal_test_f1_dt]  
}).T  
  
metrics_dt.columns = ['Decision Tree - Seasonal']  
metrics_dt
```

Out[329]:

**Decision Tree -
Seasonal**

Train Accuracy

0.757688

Test Accuracy	Decision Tree - Seasonal
Gap	0.004786
Train Recall	0.693104
Test Recall	0.674306
Train F1 Score	0.727052
Test F1 Score	0.717587

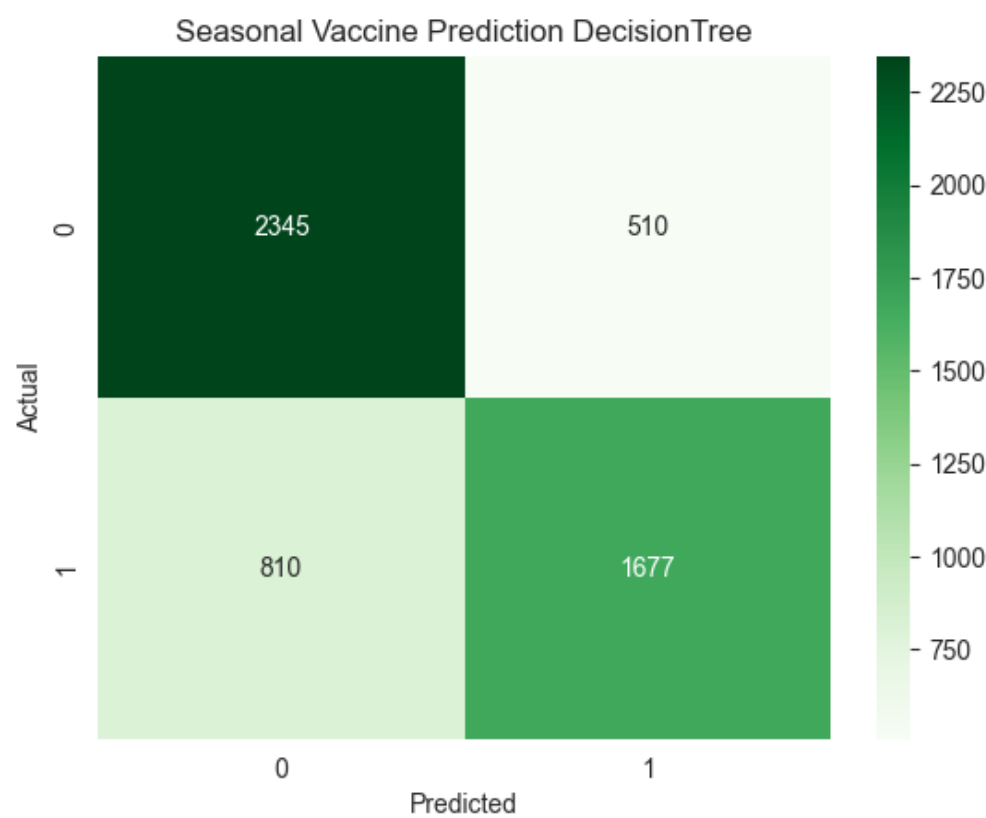
confusion matrix heatmaps

In [330]:

```
# Plot logReg confusion matrix for seasonal vaccine
sns.heatmap(confusion_matrix(y_test['seasonal_vaccine'], test_pred_seasonal_dt),
            annot=True, fmt='d', cmap='Greens')

# Set labels using plt instead of ax
plt.title('Seasonal Vaccine Prediction DecisionTree')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

plt.savefig('./Visualizations/DecisionTree_confusion_matrix.png')
```



<Figure size 640x480 with 0 Axes>

Decision tree model summary and insights

The decision tree model correctly classified 2345 non-vaccinated and 1677 vaccinated patients for the seasonal vaccine. However, it made 510 false positives and 810 false negatives, indicating a tendency to under-predict vaccination status.

overfitting or underfitting analysis:

The gap is generally small, indicating that the model generalizes reasonably well to unseen data without significant overfitting or underfitting. The slightly lower recal on test set suggests some under-prediction of vaccinated cases.

model 3

RANDOM FOREST CLASSIFIER MODEL

In [331]:

```
rf_seasonal = RandomForestClassifier(
    n_estimators=100, max_depth=5, random_state=42, class_weight='balanced')

rf_seasonal.fit(X_train, y_train['seasonal_vaccine'])

# Predictions
train_preds_seasonal_rf = rf_seasonal.predict(X_train)
test_preds_seasonal_rf = rf_seasonal.predict(X_test)

# Accuracy
seasonal_train_acc_rf = accuracy_score(y_train['seasonal_vaccine'], train_preds_seasonal_rf)
seasonal_test_acc_rf = accuracy_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf)
seasonal_gap_rf = seasonal_train_acc_rf - seasonal_test_acc_rf

# Recall
seasonal_train_recall_rf = recall_score(y_train['seasonal_vaccine'], train_preds_seasonal_rf)
seasonal_test_recall_rf = recall_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf)

# F1 Score
seasonal_train_f1_rf = f1_score(y_train['seasonal_vaccine'], train_preds_seasonal_rf)
seasonal_test_f1_rf = f1_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf)
```

In [332]:

```
metrics_rf=pd.DataFrame({
    "Train Accuracy": [seasonal_train_acc_rf],
    "Test Accuracy": [seasonal_test_acc_rf],
    "Gap": [seasonal_train_acc_rf - seasonal_test_acc_rf],

    "Train Recall": [seasonal_train_recall_rf],
    "Test Recall": [seasonal_test_recall_rf],

    "Train F1 Score": [seasonal_train_f1_rf],
    "Test F1 Score": [seasonal_test_f1_rf]
}).T

metrics_rf.columns = ['Random Forest - Seasonal']
metrics_rf
```

Out[332]:

Random Forest - Seasonal	
Train Accuracy	0.769249
Test Accuracy	0.768626
Gap	0.000623
Train Recall	0.736027
Test Recall	0.724970
Train F1 Score	0.748135
Test F1 Score	0.744734

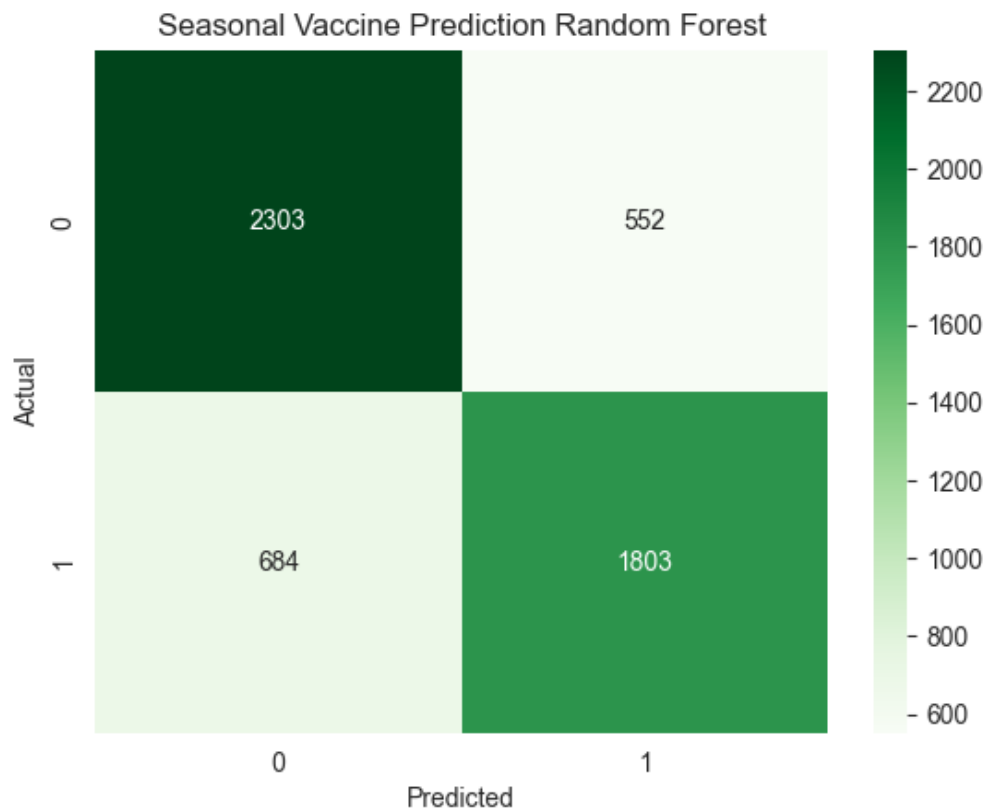
confusion matrix heatmaps

In [333]:

```
# Plot logReg confusion matrix for seasonal vaccine
sns.heatmap(confusion_matrix(y_test['seasonal_vaccine'], test_preds_seasonal_rf),
            annot=True, fmt='d', cmap='Greens')

# Set labels using plt instead of ax
plt.title('Seasonal Vaccine Prediction Random Forest')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

plt.savefig('./Visualizations/randomforest_confusion_matrix.png')
```



<Figure size 640x480 with 0 Axes>

Random forest model summary and insights

The random forest model correctly classified 2303 non-vaccinated and 1803 vaccinated patients for the seasonal vaccine. errors include 684 false positives and 552 false negatives, indicating a moderate tendency to under-predict vaccination status.

overfitting or underfitting analysis:

the small gap shows minimal overfitting, indicating good generalization to unseen data.

comparison table of model performances

In [334]:

```
# storing metrics for each model and each target
metrics = []

# Logistic Regression
metrics.append({
    "Model": "Logistic Regression - Seasonal",
```

```

    "Accuracy": accuracy_score(y_test['seasonal_vaccine'], test_pred_seasonal_lr),
    "Precision": precision_score(y_test['seasonal_vaccine'], test_pred_seasonal_lr),
    "Recall": recall_score(y_test['seasonal_vaccine'], test_pred_seasonal_lr),
    "F1-score": f1_score(y_test['seasonal_vaccine'], test_pred_seasonal_lr),
    "Gap": test_accuracy_seasonal_lr - train_accuracy_seasonal_lr,
})

# Decision Tree
metrics.append({
    "Model": "Decision Tree - Seasonal",
    "Accuracy": accuracy_score(y_test['seasonal_vaccine'], test_pred_seasonal_dt),
    "Precision": precision_score(y_test['seasonal_vaccine'], test_pred_seasonal_dt),
    "Recall": recall_score(y_test['seasonal_vaccine'], test_pred_seasonal_dt),
    "F1-score": f1_score(y_test['seasonal_vaccine'], test_pred_seasonal_dt),
    "Gap": seasonal_test_acc_dt - seasonal_train_acc_dt,
})

# Random Forest
metrics.append({
    "Model": "Random Forest - Seasonal",
    "Accuracy": accuracy_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf),
    "Precision": precision_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf),
    "Recall": recall_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf),
    "F1-score": f1_score(y_test['seasonal_vaccine'], test_preds_seasonal_rf),
    "Gap": seasonal_test_acc_rf - seasonal_train_acc_rf,
})

# Create DataFrame
results = pd.DataFrame(metrics)
results

```

Out[334]:

	Model	Accuracy	Precision	Recall	F1-score	Gap
0	Logistic Regression - Seasonal	0.779858	0.765277	0.760354	0.762808	0.004758
1	Decision Tree - Seasonal	0.752902	0.766804	0.674306	0.717587	-0.004786
2	Random Forest - Seasonal	0.768626	0.765605	0.724970	0.744734	-0.000623

In [335]:

```
results.sort_values(by="F1-score", ascending=False)
```

Out[335]:

	Model	Accuracy	Precision	Recall	F1-score	Gap
0	Logistic Regression - Seasonal	0.779858	0.765277	0.760354	0.762808	0.004758
2	Random Forest - Seasonal	0.768626	0.765605	0.724970	0.744734	-0.000623
1	Decision Tree - Seasonal	0.752902	0.766804	0.674306	0.717587	-0.004786

In [336]:

```

# 1. Prepare data
metrics_list = ['Accuracy', 'Precision', 'Recall', 'F1-score']
df_melted = results.melt(
    id_vars='Model', value_vars=metrics_list, var_name='Metric',
    value_name='Score')

# 2. Plotting - Create two subplots (2 rows, 1 column)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10), gridspec_kw={'height_ratios': [3, 1]})
sns.set_style("whitegrid")

# --- TOP CHART: Main Metrics ---

```

```

sns.barplot(
    data=df_melted, x='Metric', y='Score', hue='Model', palette='viridis',
    ax=ax1)
ax1.set_title('Performance Comparison: Seasonal Models',
              fontsize=15, fontweight='bold')
ax1.set_ylim(0.65, 0.85)
ax1.set_ylabel('Score Value')
ax1.legend(title='Model Type', bbox_to_anchor=(1.05, 1), loc='upper left')

# Add values on top of bars for main metrics
for p in ax1.patches:
    ax1.annotate(format(p.get_height(), '.3f'),
                  (p.get_x() + p.get_width() / 2., p.get_height()),
                  ha='center', va='center', xytext=(0, 9), textcoords='offset points', font
tsize=9)

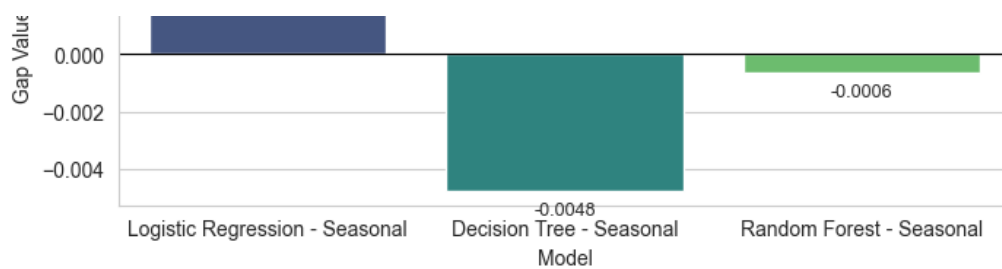
# --- BOTTOM CHART: Gap ---
sns.barplot(data=results, x='Model', y='Gap', palette='viridis', ax=ax2)
ax2.set_title('Gap Comparison (Train vs Test)', fontsize=12, fontweight='bold')
ax2.set_ylabel('Gap Value')
ax2.axhline(0, color='black', linewidth=0.8) # Add a line at zero

# Add values on top of gap bars
for p in ax2.patches:
    ax2.annotate(format(p.get_height(), '.4f'),
                  (p.get_x() + p.get_width() / 2., p.get_height()),
                  ha='center', va='bottom' if p.get_height() > 0 else 'top',
                  xytext=(0, 5 if p.get_height() > 0 else -5),
                  textcoords='offset points', fontsize=9)

plt.tight_layout()
plt.savefig('./Visualizations/Model_comparison_chart_with_gaps.png')
plt.show()

```





In [337]:

```
from sklearn.metrics import PrecisionRecallDisplay
import matplotlib.pyplot as plt

# Create the plot figure
fig, ax = plt.subplots(figsize=(10, 7))

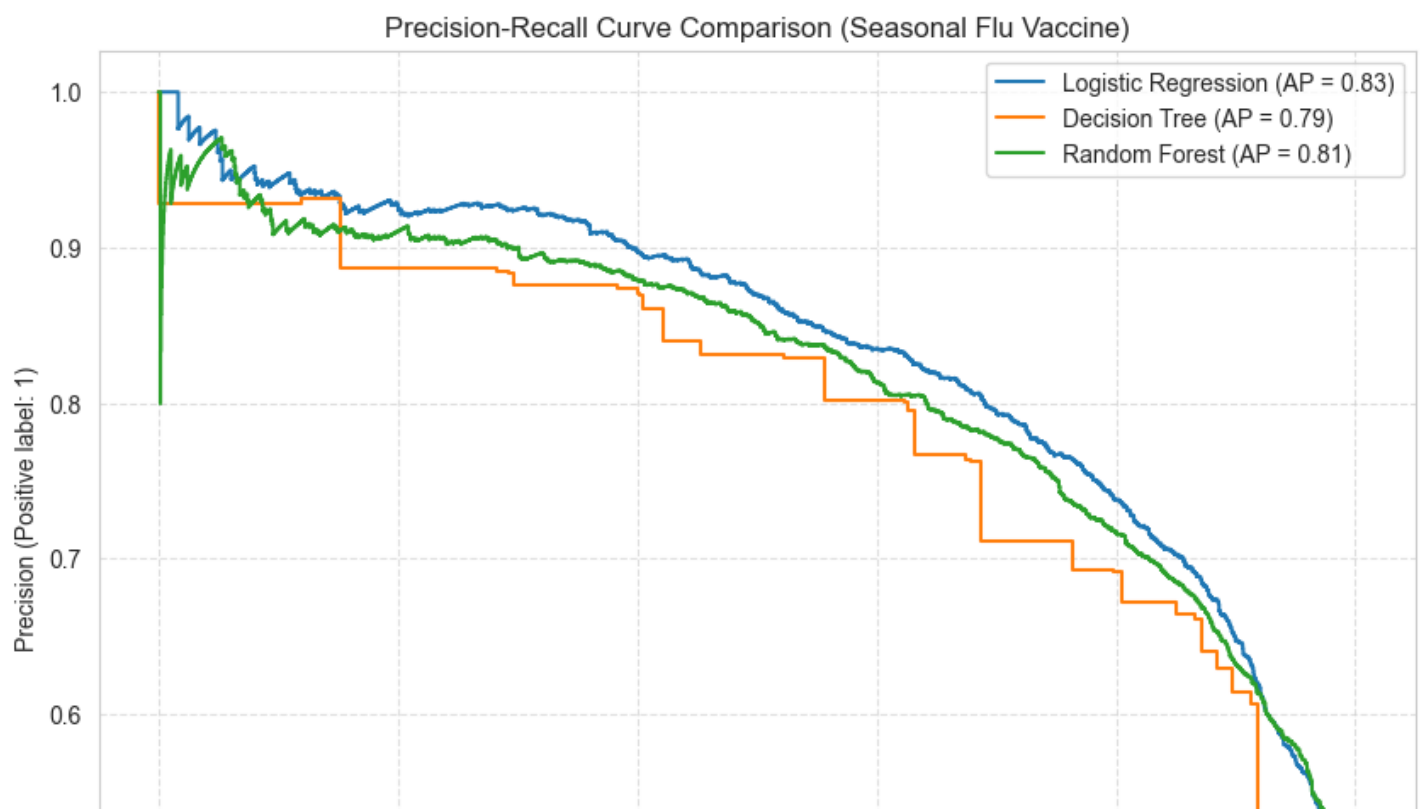
# 1. Plot Logistic Regression PR Curve
# Note: uses X_test_scaled as it was trained on scaled data
PrecisionRecallDisplay.from_estimator(
    logReg_seasonal, X_test_scaled, y_test['seasonal_vaccine'],
    ax=ax, name="Logistic Regression"
)

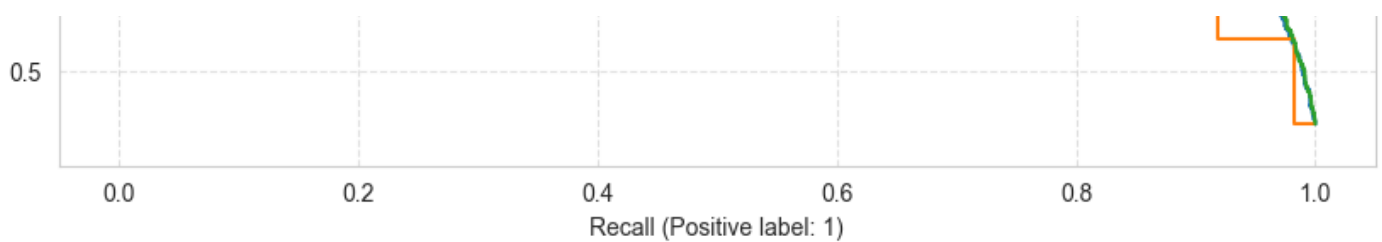
# 2. Plot Decision Tree PR Curve
PrecisionRecallDisplay.from_estimator(
    dt_seasonal, X_test, y_test['seasonal_vaccine'],
    ax=ax, name="Decision Tree"
)

# 3. Plot Random Forest PR Curve
PrecisionRecallDisplay.from_estimator(
    rf_seasonal, X_test, y_test['seasonal_vaccine'],
    ax=ax, name="Random Forest"
)

# Customize the plot
ax.set_title("Precision-Recall Curve Comparison (Seasonal Flu Vaccine)")
plt.legend(loc="upper right")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

# Save the visualization
plt.savefig('./Visualizations/precision_recall_comparison.png')
```





<Figure size 640x480 with 0 Axes>

Key Findings

1. Overall accuracy

Logistic regression performs better in overall accuracy(78%) compared to decision tree(75%) and random forest(77%) for the seasonal flu vaccine prediction.

2. Recall vs precision

-Decision Tree has the highest precision(76.68), meaning it is slightly better at correctly predicting vaccinated patients, which is crucial in public health contexts to ensure high vaccination coverage. -Logistic regression has a balanced precision(76%) and recall(76%), showing good general performance -Decision tree lags behind in recall metrics(67%), indicating it may not be the best choice for this task.

3. F1-score

BValances both precion and recall.

Logistic regression(76%)> Random Forest(75%) > Decision Tree(72%) for seasonal vaccine prediction. Indicating logistic regression is the best overall model for this task.

4. **Overfitting/Underfitting Analysis** All models show small gaps (less that 1%) hence minimal overfitting and underfitting. Random forest slightly overfits compared to logistic regression but the difference is negligible.

5. **Precision Recall Curve** The perfect model will be the top right one(Logistic Regression Model), meaning it caught 100% of vaccine recipients (recall) and were 100% correct on every single one of them (precision) The **Average Precision(AP)** represents area under the curve and the Logistic Regression Model with a score of ~0.76 is strong and indicates the model did a good job balancing the two competing goals

Final Model secection

For the **Seasonal Flu Vaccine** campaign , Logistic Regression is the recommended model.

In vaccine outreach, you want to identify as many potential vaccine recipients as possible (High Recall) while maintaining a high level of confidence in those predictions (High Precision). Logistic Regression achieves a 76% balance for both, whereas the tree-based models would cause you to "miss" nearly 10\% more of the target population.

In [338]:

```
risk_by_age = df.groupby('age_group')['opinion_seas_risk'].mean()
print(risk_by_age)
```

```
age_group
18 - 34 Years    2.640652
35 - 44 Years    2.692308
45 - 54 Years    2.656739
55 - 64 Years    2.758224
65+ Years        2.756101
Name: opinion_seas_risk, dtype: float64
```

In [339]:

```
# Run this to compare 'Risk' vs 'Trust'
opinion_stats = df.groupby('age_group')[['opinion_seas_risk', 'opinion_seas_vacc_effecti
ve']].mean()
print(opinion_stats)
```

```
age_group    opinion_seas_risk    opinion_seas_vacc_effective
```

age_group	Perceived Risk of Flu	Trust in Vaccine Effectiveness
18 - 34 Years	2.640652	3.858293
35 - 44 Years	2.692308	3.875520
45 - 54 Years	2.656739	3.942917
55 - 64 Years	2.758224	4.090599
65+ Years	2.756101	4.247698

In [340]:

```
# Calculate the stats directly from your dataframe
opinion_stats = df.groupby('age_group')[['opinion_seas_risk', 'opinion_seas_vacc_effective']].mean()

# Reset index and melt to prepare for Seaborn plotting
plot_df = opinion_stats.reset_index().melt(id_vars='age_group',
                                          var_name='Metric',
                                          value_name='Average Score')

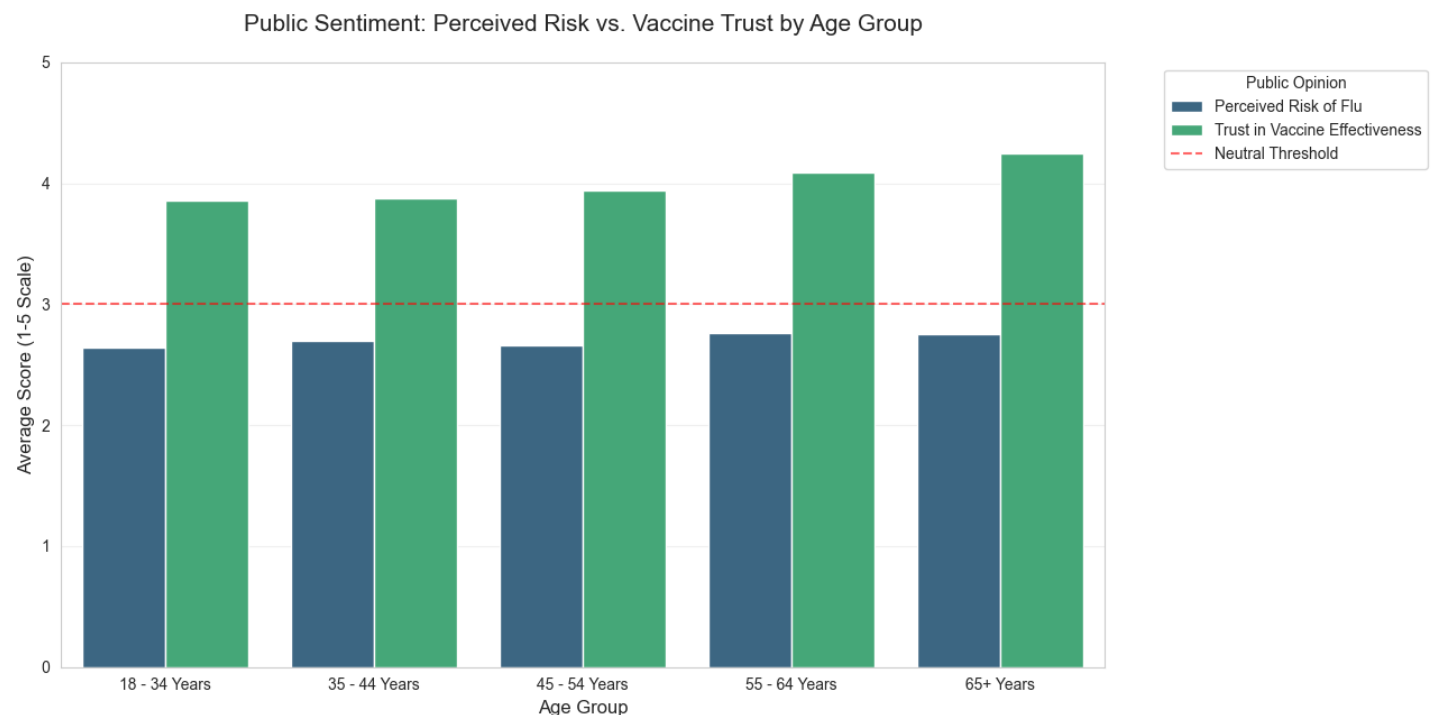
# Rename metrics for a cleaner legend
plot_df['Metric'] = plot_df['Metric'].replace({
    'opinion_seas_risk': 'Perceived Risk of Flu',
    'opinion_seas_vacc_effective': 'Trust in Vaccine Effectiveness'
})

# Create the visualization
plt.figure(figsize=(12, 7))
sns.barplot(data=plot_df, x='age_group', y='Average Score', hue='Metric', palette='viridis')

# Add a reference line for "Neutral" (3.0 on a 1-5 scale)
plt.axhline(3.0, color='red', linestyle='--', alpha=0.6, label='Neutral Threshold')

# Formatting
plt.title('Public Sentiment: Perceived Risk vs. Vaccine Trust by Age Group', fontsize=15, pad=20)
plt.ylabel('Average Score (1-5 Scale)', fontsize=12)
plt.xlabel('Age Group', fontsize=12)
plt.ylim(0, 5)
plt.legend(title='Public Opinion', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', alpha=0.3)

plt.show()
plt.savefig('./Visualizations/public_sentiment.png')
```



<Figure size 640x480 with 0 Axes>

What drives vaccination?

We visualize the top 10 features that influence model predictions. These features represent the most critical leverage points for increasing vaccine uptake

In [341]:

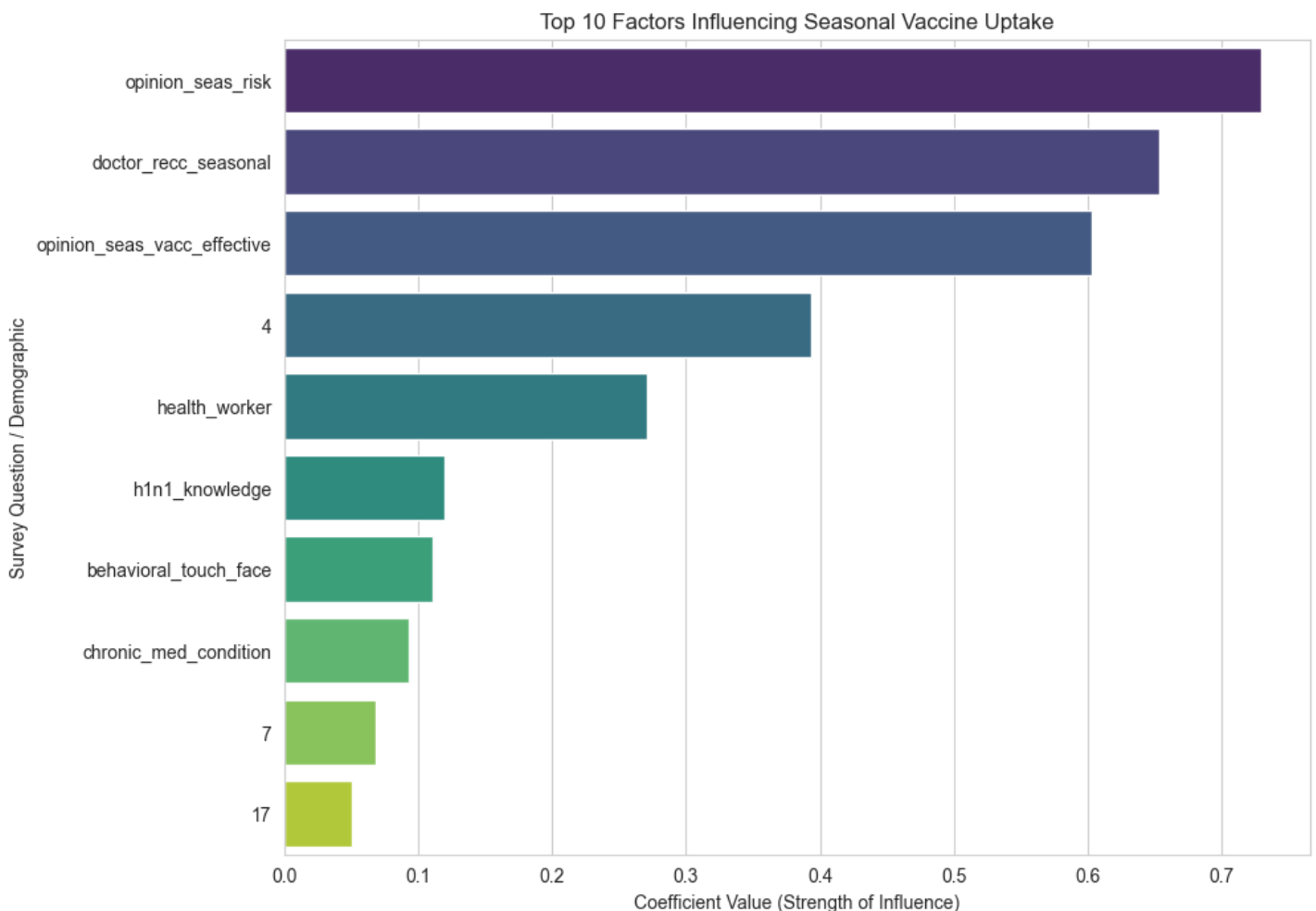
```
coefficients = logReg_seasonal.coef_[0]
feature_names = X_train.columns

# 2. Creating a DataFrame for easy viewing
feat_importance = pd.DataFrame({'Feature': feature_names, 'Coefficient': coefficients})

# 3. Sorting by the strongest positive impact
feat_importance = feat_importance.sort_values(by='Coefficient', ascending=False)

# 4. Plot the top features
plt.figure(figsize=(10, 8))
sns.barplot(data=feat_importance.head(10), x='Coefficient', y='Feature', palette='viridis')
plt.title('Top 10 Factors Influencing Seasonal Vaccine Uptake')
plt.xlabel('Coefficient Value (Strength of Influence)')
plt.ylabel('Survey Question / Demographic')
plt.show()

plt.savefig('./Visualizations/factors_influencing_uptake.png')
```



<Figure size 640x480 with 0 Axes>

Recommendations For stakeholders

1. **prioritize individuals with low predicted uptake.** From the public sentiment we observe that 18-34 agegroup is the lowest point of engagement and vaccination rates, we need to raise this bar.
2. **Target outreach campaigns** since the model has high Recall(76%), use predicted probabilities to identify individuals with low likelihood of vaccination.
3. **launch severity awareness campaigns**-highlight statistics of people hospitalized by the flu each year. Focus on severity of the flu and risk perception(how easy it is to catch the flu) because the

opinion_seas_vacc_effective is low and opinion_seas_risk is high since people are likely to get vaccinated if they feel the risk.

4. shift focus of campaigns from *where to get vaccine* to *why the flu is a serious risk* specifically targetting 18-24 and 65+ age groups.
5. social Media Engagements-Launch severity awareness campaigns on social media platforms to specifically reach the "younger" demographic (18–34), where public sentiment is currently lowest.

Limitations

- The model is based on self-reported survey data, which may contain bias or inaccuracies, especially for health behaviors and vaccine beliefs.
- Prioritizing recall increased false positives, meaning some individuals identified as unlikely to vaccinate may not actually require intervention.
- The data reflects a specific time period and public health context, which may limit the model's ability to generalize to future flu seasons or different populations.